

Chapter Two

GUI

GUI

- ê Stands for Graphical User Interface ('Goo-ee')
- ê Pictorial interface to a program
 - Distinctive 'look' and 'feel'
- ê Different applications with consistent GUIs improve productivity
 - Example:
 - Menu bar, text field, label etc.
- ê Are built from Containers and components
- ê Allows us to interact with our programs through mouse movements, button clicks, key presses, and so on.

ê There are three basic things that are necessary to develop a graphical user interface

- **The GUI components and containers sets** : - form the interface
 - panels, frames, buttons, labels, check boxes, etc
- **Component arrangement (Layout)**: - the scheme where by the UI components are arranged to create the interface
- **Response to user requests**: - the act of associating actions to user requests, known as 'events'

Packages

- ê The java.awt package contains the core AWT graphics classes:
- ê GUI Component classes, such as Button, TextField, and Label.
- ê GUI Container classes, such as Frame and Panel.
- ê Layout managers, such as FlowLayout, BorderLayout and GridLayout.
- ê Custom graphics classes, such as Graphics, Color and Font.
- ê **The java.awt.event package supports event handling:**
- ê Event classes, such as ActionEvent, MouseEvent, KeyEvent and WindowEvent,
- ê Event Listener Interfaces, such as ActionListener, MouseListener, MouseMotionListener, KeyListener and WindowListener,
- ê Event Listener Adapter classes, such as MouseAdapter, KeyAdapter, and WindowAdapter.

Implementing GUIs in Java

- ê The Java Foundation Classes (JFC) Encompass a group of features for constructing graphical user interfaces (GUI).
- ê JFC encompasses the following APIs:
 - **Abstract Window Toolkit (AWT)**: native GUI components
 - **Swing**: light weight GUI components
 - **2D**: rendering two-dimensional shapes, text, and images
 - **Accessibility**: allowing compatibility with, for example, screen readers and screen magnifiers
 - **Internationalization**: allows developers to build applications that can interact with users worldwide in their own languages and cultural conventions

Using AWT Components

ê AWT is Java's original set of classes for building GUIs

Uses peer components of the OS;

e.g java.awt.Button class has a peer named java.awt.peer.Button

Heavyweight (powerful, influential)

Not truly portable: looks different and lays out

Inconsistently on different OSs

- Due to OS's underlying display management system

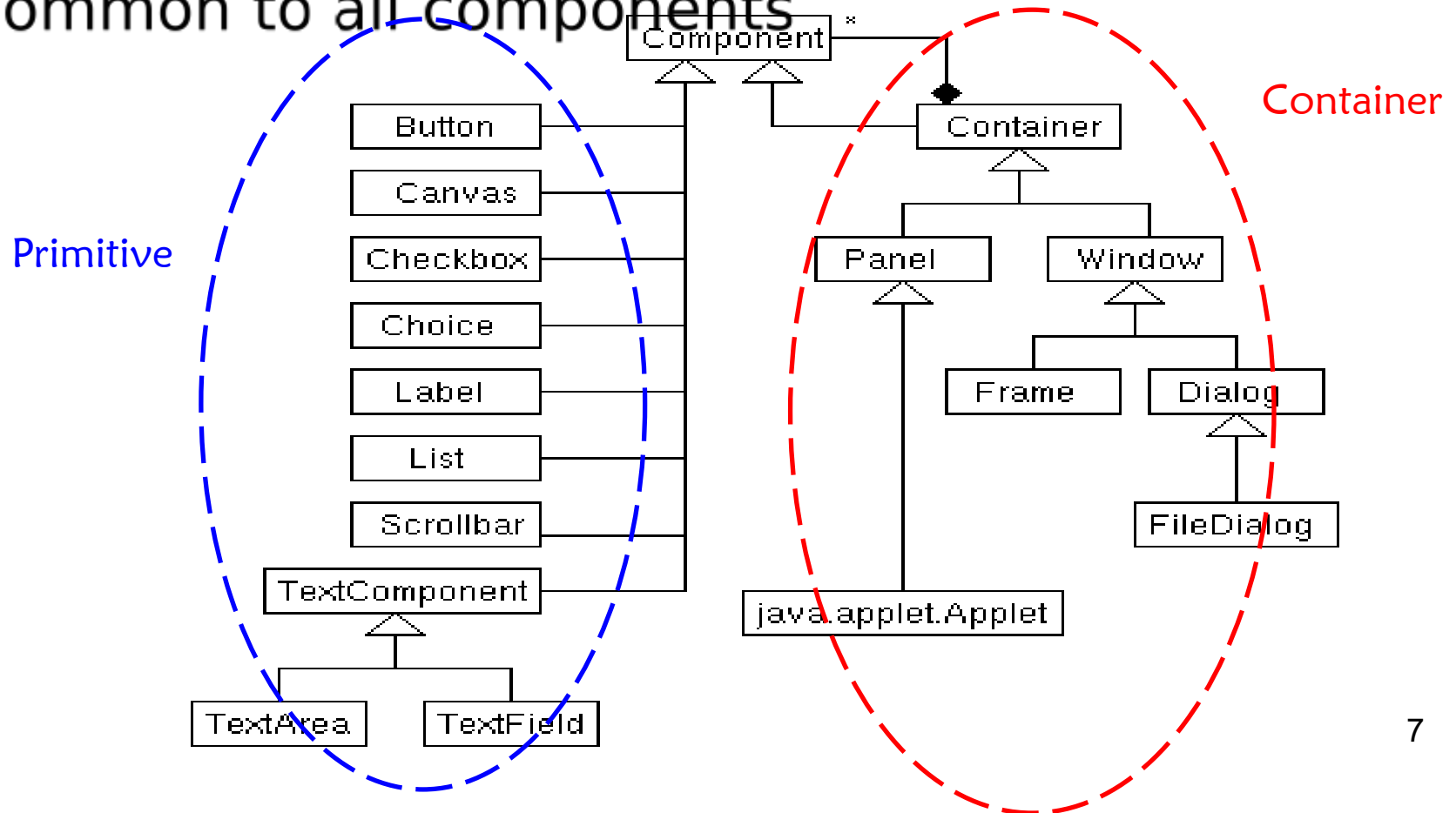
ê Every GUI component is a subclass of the AWT

Component class (except for **menus**)

- Controls: - allow simple user input
- Labels: - display text
- Containers: - arrange other components
- Use Canvases: - for custom components

Components and Containers

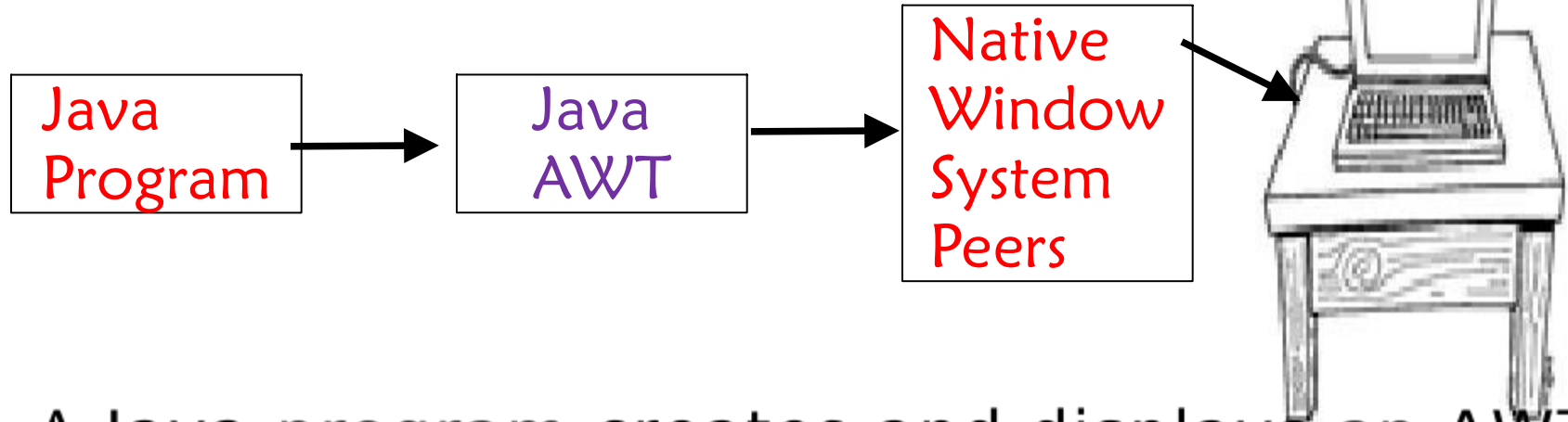
- ê The AWT set is structured in a class hierarchy
- ê Every UI component is a descendant of the **Component** class
- ê This contains the very basic functionality common to all components



Peers and Platform Independence

- ê UI components that have peers are called *heavyweight because*
 - They are rendered in their own (opaque - not transparent or, hard to understand) windows and thus are expensive to use,
 - They must be rectangular and cannot have transparent backgrounds, and
 - They are not willing to being sub-classed

Using Peers



- A Java program creates and displays an AWT component,
- Java AWT creates and displays a native component, or peer.
- By using peers
 - Platform controls component appearance
 - Inconsistencies in implementations
 - Interfacing to native platform error-prone

AWT Limitations

- ê The AWT defines a basic set of controls, windows, and dialog boxes that support a usable, but limited graphical interface
- ê **One reason for the limited nature of the AWT is that it translate its various visual components into their corresponding, platform-specific equivalents or peers**
- ê **This means that the look and feel of a components is defined by the platform, not by java.** Because the AWT components use native code resources, they are refer to as heavy weight
- ê Fewer component types: tables and trees these important components are missing. They are commonly used in desktop application

AWT Limitations....

- ê Lack of rich component features: The button does not support pictures
- ê No extensibility: It is not possible to inherit and reuse an existing AWT components
- ê The use of native peers led to several problems. First, because of variations between operating systems, a component might look, or even act, differently on different platforms
- ê The look and feel of each component was fixed and could not be changed

AWT 1.1. Lightweight Components

- ê AWT 1.1 introduced the notion of *lightweight* components which:
 - Are contained within a heavyweight component's window
 - Does not have peers
 - Are rendered in their container's window rather than one of their own
 - Do not incur performance penalties and can have transparent backgrounds
- ê Almost all Swing components are lightweight ones that extend either
 - **java.awt.Component** or **java.awt.Container**

What is Swing ?

- ê The **Swing toolkit** includes a rich set of components for building GUIs and adding interactivity to Java applications
- ê **Swing** includes all the components you would expect from a modern toolkit:
 - table controls, list controls, tree controls, buttons, and labels
- ê **Swing** supports numerous look and feels, including the ability to create your own look and feel
- ê **Swing** is part of the Java Foundation Classes (JFC)
- ê **Features**
 - **Pluggable Look and Feel:** - allows a program to have control over its appearance.
 - **Data Transfer:** - Most programs will want to use drag and drop or cut, copy and paste

ê **importing packages:**

```
import java.awt.*;  
import javax.swing.*;
```

ê **Swing Philosophy**

- Richer Component Set
 - Replaces AWT Component Set
 - Adds more complex components
- Swing Components Java-Based

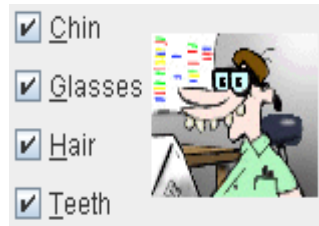
ê **The Swing Containment Hierarchy:** - typically has at least:

- **Top-Level Container:** the root container that holds everything together. e.g. **JApplet, JFrame, Window**
- **Intermediate Container:** a container to simplify the positioning of atomic components. e.g. **JPanel, JScrollPane**
- **Atomic Component:** a self-sufficient component not holding other components. eg. **JButton, JLabel**

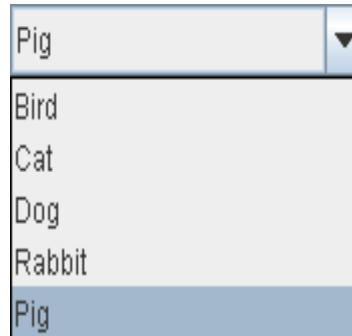
A Visual Guide to Swing Components



JButton



JCheckBox



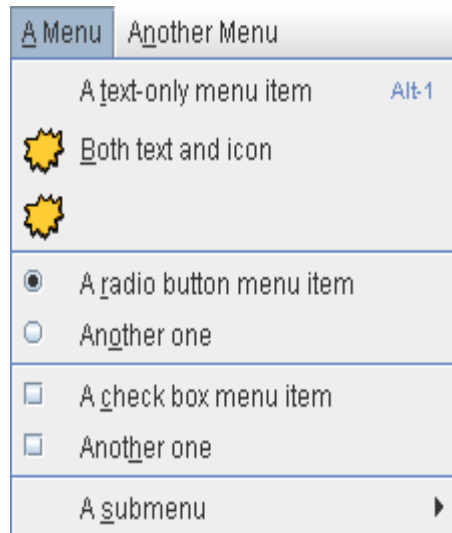
JComboBox



JList



JTextField



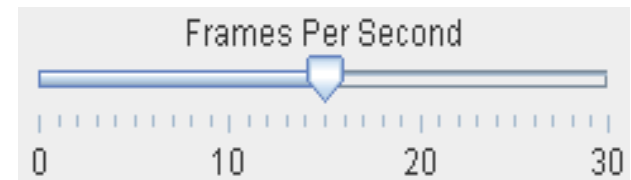
JMenu



JSpinner



JPasswordField



JSlider



JLabel

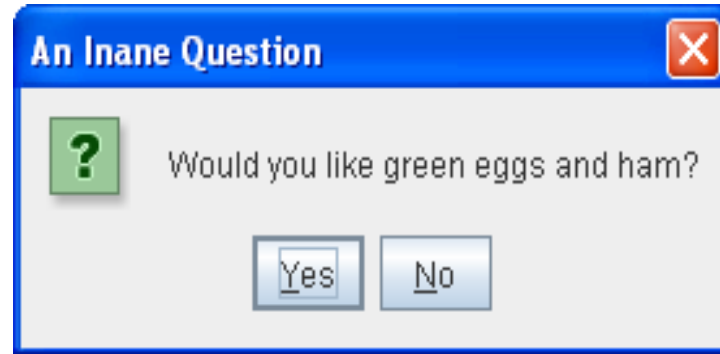


JRadioButton

★ Top-Level Containers



JApplet

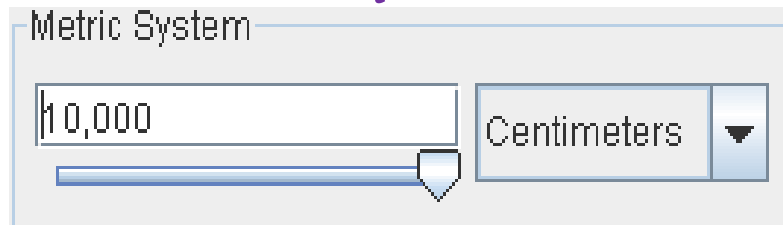


JDialog

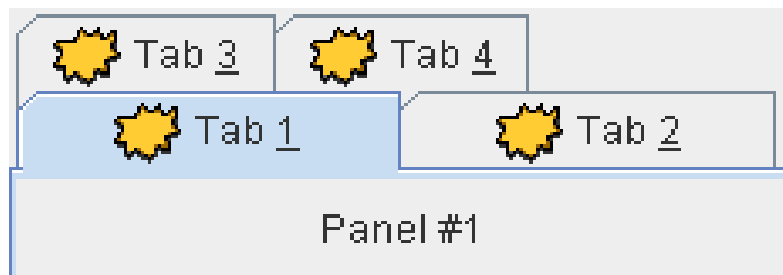


JFrame

★ General-Purpose Containers



JPanel

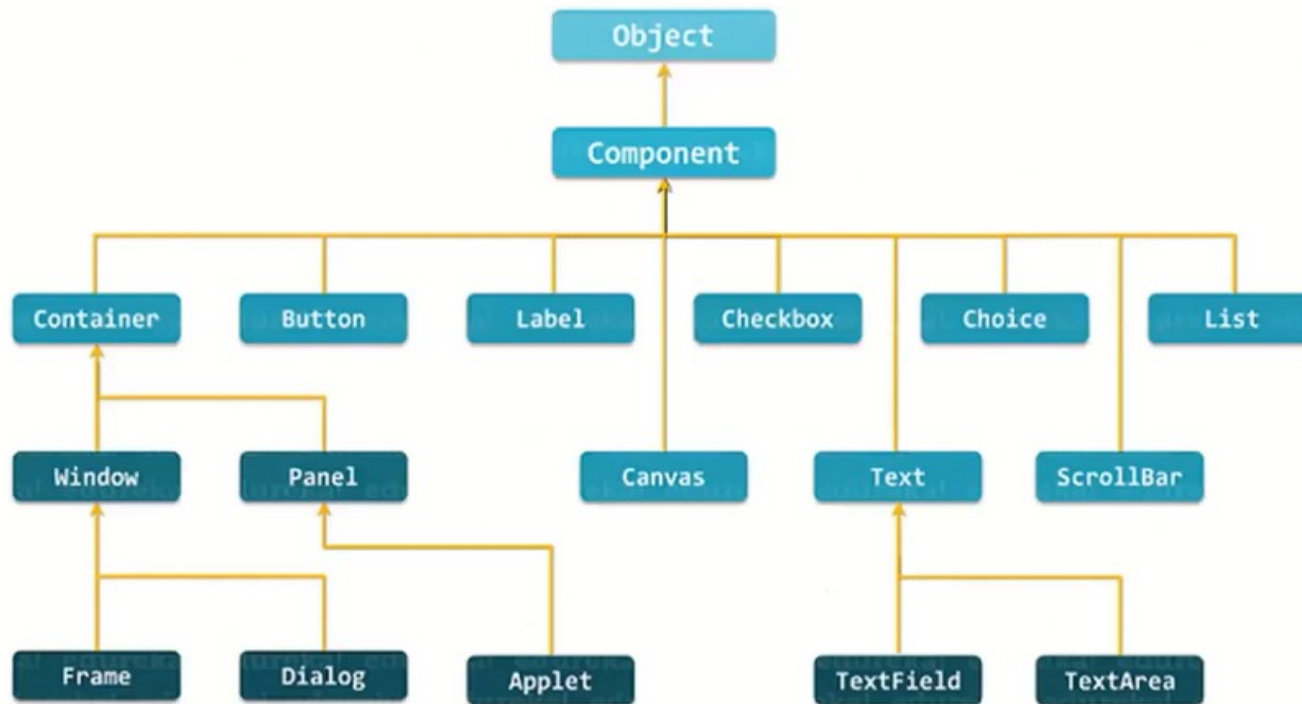


JTabbedPane



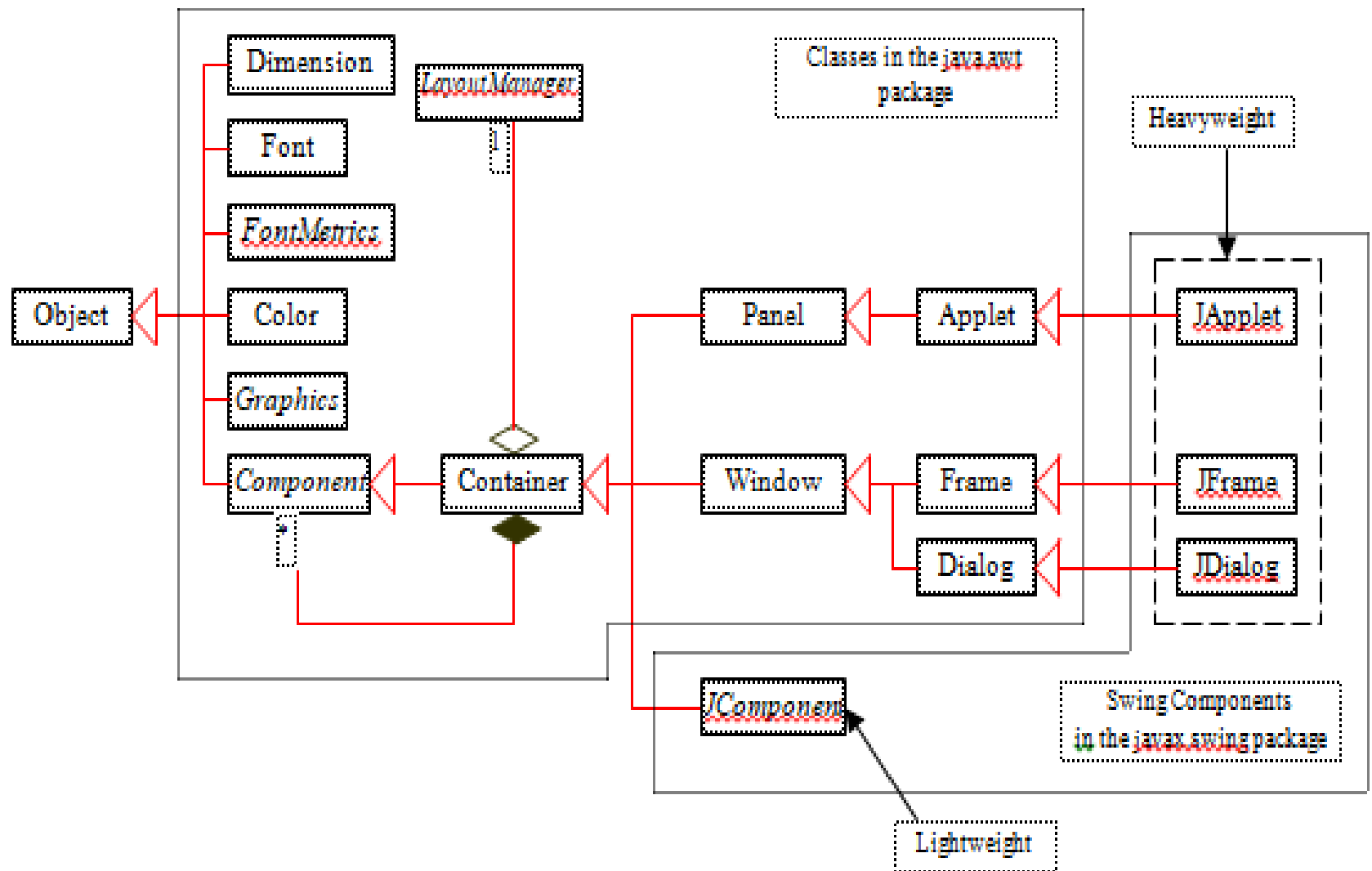
JScrollPane

Components and Containers



- A Container is a special type of component
- You can add other components. Example, JFrame, JApplet, JPanel etc
- All containers have methods to **add** and **remove** components

GUI Class Hierarchy (Swing)



Description of Classes

- ê **Object**: All classes ultimately derive from Object, thus this class is at the top of the tree.
- ê **Component**: represents an object that has a visual representation that can be shown on-screen and that can interact with users. This class defines some basic methods that are available to all Swing classes.
- ê **Container**: builds on the basic visual capabilities of the Component class by adding the ability to hold other containers.
- ê **Window**: a specialized type of container object that has a border, a title bar, buttons that minimize, maximize, and close the window, and that can be repositioned and possibly even resized by the user.

- ê **Frame**: a type of Window that serves as the basis for Java GUI applications. Frame is an AWT class that has been improved upon by the JFrame class.
- ê **JFrame**: the Swing version of the older Frame class. Most of the Swing applications include at least one JFrame object.
- ê **JComponent**: is the basis for all other Swing components except for frames.
- ê **JPanel**: used to organize and control the layout of other components such as labels, buttons, text fields, etc. In most Swing applications, one or more panels are added to a frame. Then, when the frame is displayed, the components that were added to its panels are made visible.
- ê **JLabel**: creates a label that displays a simple text value.

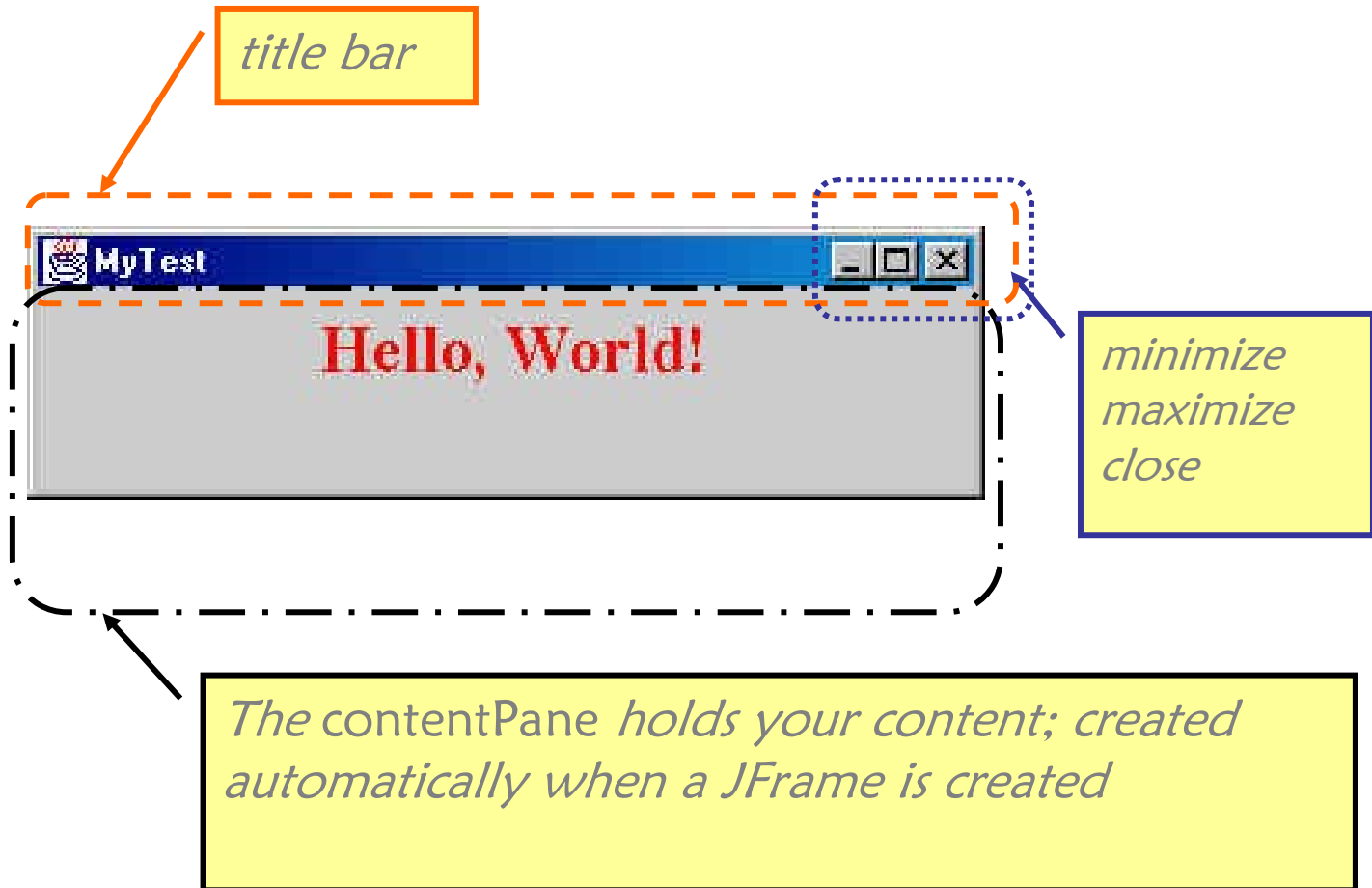
AWT vs. Swing

- ê Swing does not replace the AWT; it is built on top of it
- ê All 1.0 AWT components are heavyweight; corresponding Swing components are lightweight
- ê Swing component names begin with “J”:
 - Component (AWT) vs. JComponent (Swing)
 - Button (AWT) vs. JButton (Swing)
- ê Always use Swing components;
- ê however, since Swing is built on top of AWT, you will need to know some AWT methods

ê **Frames**

- Frame is a window that is not contained inside another window.
- Frame is the basis to contain other user interface components in Java GUI applications.
- **JFrame** is the application window class
- The **JFrame** class is special; it draws the window and interacts with the operating system
- When a JFrame is created, an inner container called the **contentPane** is automatically created
- We don't draw graphics directly on JFrame; we draw on the **contentPane**

Anatomy of a JFrame



Run Java code from CMD....

8. **Paste the path that you copied in step 1 and then click on OK button at the bottom of the window**
9. **Click on New Button, which is at the bottom of User variables for “name of your computer account” panel of Environmental Variables window**
10. **In the text fields of New User Variable Window, write the word “path” in the variable name text field and paste the path that you copied in step 1 in the variable Value text field, then click on OK button.**
11. **Click on OK button of Environmental variable Window**
12. **Click on OK button of System Properties Window**
13. **Open CMD and change the directory to the folder where your java source code is found**

Run Java code from CMD....

14. **Open CMD and change the directory to the folder where your java source code is found**

e.g

```
C:\Users\Admin>cd C:\Users\Admin\Desktop\AP\Java Codes
```

15. **Write “ javac filename.java”, in the coming cmd line to compile the code**

N.B, the file name should be the name of the class in your java code

16. **If there is no error in the compiled source code, type “java filename”**

and press enter to see the out put

Example: Empty Frame

```
package swinglab;

import java.awt.*;
import javax.swing.*;
// extends keyword makes Calc a JFrame
public class Calc extends JFrame{
    public Calc() {
        // get the contentPane and assign it to cp
        Container cp = getContentPane();
        // exit program when user closes the window
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        // sets the layout; will be covered in later slide
        cp.setLayout(new FlowLayout());
        // sets title to "My Funky Calculator"
        setTitle("My First Calculator");
        setSize(1000,700); // Frame has 0 default size
    }
}
```

```
public static void main (String[] args){  
    Calc trial = new Calc();  
    // Frame is invisible by default  
    trial.setVisible(true);  
  
    // main method exits but user interface  
    // stays alive  
}  
}
```

Using the JPanel Class

- ê A panel is a type of container
- ê Designed to hold a group of components
- ê The normal way to display a group of controls is to add those controls to a panel, and then add the panel to the frame
- ê You can bypass the panel and add the controls directly to the frame if you want.
- ê **Constructor**
- ê **JPanel ()** : -Creates a new panel
- ê **JPanel(LayoutManager layout)**: - Creates a new panel with the specified layout manager. The default layout manager is **FlowLayout**

Method

- ê **void add (Component c):** - Adds the specified component to the panel.
- ê **void remove (Component c):** - Removes the specified component from the panel.
- ê **void setLayout (LayoutManager layout):** - Sets the layout manager used to control how components are arranged when the panel is displayed.
- ê **void setLocation (int x, int y):** - Sets the x and y position of the frame-screen. The top-left corner of the screen is 0, 0.
- ê **void setSize (int width, int height):** - Sets the size of the frame to the specified width and height.

Example:

```
//package swinglab;

import java.awt.*;
import javax.swing.*;

public class Calc extends JFrame{
    private JPanel entryPanel;
    private JPanel answerPanel;
    public Calc() {
        Container cp = getContentPane();
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        cp.setLayout(new FlowLayout());
        cp.setBackground(Color.white);
        setTitle("My Funky Calculator");
        setSize(1000,700);
        entryPanel = new JPanel();
        entryPanel.setBackground(Color.orange);
        answerPanel = new JPanel();
        answerPanel.setBackground(Color.yellow);
        // . . .
    }
}
```

```
        cp.add(entryPanel);  
        cp.add(answerPanel);  
    }  
    public static void main (String[] args){  
        Calc trial = new Calc();  
        trial.setVisible(true);  
    }  
}
```

JComponent

- ê **JComponent**: The base class for all Swing components except top-level containers
- ê **JComponents** do not extend their AWT counterparts:
 - For example, the JButton class is not a subclass (direct or indirect) of Button
- ê Note that JComponents are containers
- ê Jcomponent's subclasses present information or interact with the user
 - **Examples**: labels (JLabels), buttons (JButtons), textfields (JTextField)

Using Labels

- ê A label is a component that simply displays text.
- ê Labels are used for a variety of purposes:
 - to display captions for other controls such as text fields or combo boxes,
 - to display informational messages, or
 - to show the results of a calculation or a database lookup.
 - to display an image, or it can display both an image and some text.
 - And you have complete control over the appearance of the text.
- ê You can specify the font, size, whether the text is bold, italic, or underlined, what color the text is displayed as, and so on.

ê **Constructor**

- **JLabel** (): - Creates a new label with no initial text.
- **JLabel** (String): - Creates a new label with specified text

ê **Methods**

- **String getText** (): - Returns the text displayed by the label.
- **void setText** (**String text**): - Sets the text displayed by the label.

```
import java.awt.*;  
import javax.swing.*;  
  
public class MyTest extends JFrame {  
    JLabel myLabel = new JLabel("Hello, World!");  
  
    public MyTest() {  
        super("MyTest");  
        setSize(350, 100);  
        getContentPane().add(myLabel);  
        setDefaultCloseOperation(EXIT_ON_CLOSE);  
        setVisible(true);  
    }  
    public static void main (String args[]) {  
        MyTest m = new MyTest();  
    }  
}
```

Simple Swing Application



Creating Buttons

- ê JButton is the most commonly used component and used to create a button the user can click.
- ê You can either create an empty button or a button with text.

ê Constructor

- **JButton ()**: - Creates a new button with no initial text.
- **JButton (String text)**: - Creates a new button with the specified text.

Methods

- **String getText ()**: - Returns the text displayed by the button.
- **void setEnabled (boolean value)**: - Enables or disables the button. The default setting is true (enabled).
- **void setText (String text)**: - Sets the text displayed by the button.
- **void setVisible (boolean value)**: - Shows or hides the button. The default setting is true (the button is visible).

JTextField - JPasswordField

ê They are single-line areas in which text can be entered by the user from the keyboard or text can simply be displayed

ê Constructors

- **JTextField(int columns)**: - Creates an empty text field with the specified number of columns.
- **JTextField(String text)**: - Creates a text field initialized with the specified text.
- **JTextField(String text, int columns)**: - Creates a text field initialized with the specified text and the column size.

Methods

- **int getColumns()**: - Returns the number of columns of the text field
- **setColumns(int)**: - Sets the number of columns in this text field. The length of the text field is changeable.
- **getText()**: - Returns the string from the text field.
- **setText(String text)**: - Puts the given string in the text field.
- **setEditable(boolean editable)**: - Enables or disables the text field to be edited. By default, editable is true.

ê Example

- `TextField textField1 = new TextField('This is the initial text');`
- `TextField textField2 = new TextField('Initial text', columns);`
- `textField.setText("hi");`

JPasswordField

- a special kind of text field

ê Constructor

- **JPasswordField(String text, int columns)** - constructs a new password field.

ê Methods

- void setEchoChar(char echo) - sets the echo character for this password field. A value of 0 resets the echo character to the default.
- Char[] getPassword() - returns the text contained in this password field.

- **JTextArea**: a component used to accept multiple line text.

Constructors

- **JTextArea()** - Default constructor - Create an empty text area
- **JTextArea(int rows, int columns)** - Creates a text area with the specified number of rows and columns.
- **JTextArea(String s, int rows, int columns)** - Creates a text area with the initial text and the number of rows and columns specified.

ê **JComboBox**

- drop-down list of items from which the user can makes selection by clicking an item or possibly by typing into the box
- *Constructor*
 - `JComboBox(String str[]);`
- *Methods*
 - `getSelectedItem();` //returns the selected item
 - `getSelectedIndex();`
 - `setMaximumRowCount(int n)` - maximum of n items will be displayed if there are more items adds scrollbar.

```
import java.awt.*;  
import javax.swing.*;  
import java.util.*;  
  
public class Calc extends JFrame{  
  
    private JLabel letLabel;  
    private JLabel answerLabel;  
    private JTextField num1;  
    private JTextField num2;  
    private JComboBox operation;  
    private JButton calculate;  
    private JButton quit;  
    private JPanel entryPanel;  
    private JPanel answerPanel;  
  
    static final String ADD_OP = "ADDITION";  
    static final String SUB_OP = "SUBTRACTION";  
    static final String MUL_OP = "MULTIPLICATION";  
    static final String DIV_OP = "DIVISION";
```

```
private static final int XSIZE = 1000;  
private static final int YSIZE = 700;  
  
public Calc() {  
    Container cp = getContentPane();  
        setDefaultCloseOperation(EXIT_ON_CLOSE);  
    cp.setLayout(new FlowLayout());  
    cp.setBackground(Color.WHITE);  
    setTitle("My Funky Calculator");  
    setSize(XSIZE,YSIZE);  
  
        entryPanel = new JPanel();  
        entryPanel.setBackground(Color.ORANGE);  
  
answerPanel = new JPanel();  
    answerPanel.setBackground(Color.YELLOW);  
  
letLabel = new JLabel("Let's Calculate!");  
    entryPanel.add(letLabel);  
    letLabel.setForeground(Color.GREEN);
```

```
num1 = new JTextField("1st Number", 10);  
entryPanel.add(num1);  
num1.setBackground(Color. LIGHT_GRAY);
```

```
num2= new JTextField("2nd Number", 10);  
entryPanel.add(num2);  
num2.setBackground(Color.LIGHT_GRAY);
```

```
operation = new JComboBox();  
operation.addItem(ADD_OP);  
operation.addItem(SUB_OP);  
operation.addItem(MUL_OP);  
operation.addItem(DIV_OP);  
entryPanel.add(operation);  
operation.setBackground(Color.BLUE);
```

```
answerLabel = new JLabel("Answer");
```

```
entryPanel.add(answerLabel);  
answerLabel.setForeground(Color.red);
```

```
calculate = new JButton("Calculate");  
calculate.setBackground(Color.pink);  
answerPanel.add(calculate);
```

```
quit = new JButton("Quit");  
answerPanel.add(quit);  
Color quitter = new Color(50,100,50);  
quit.setBackground(quitter);
```

```
cp.add(entryPanel);  
cp.add(answerPanel);
```

```
}  
public static void main(String[] args){  
    Calc trial = new Calc();  
    trial.setVisible(true);  
}
```

```
}
```

Let's Calculate!

Exercise

- ê Write a program that creates the following image like window

The image shows a Java Swing window titled "This is my Form". The window has a standard Mac OS-style title bar with a small icon on the left and minimize, maximize, and close buttons on the right. The main content area is light gray and contains the following elements:

- A label "This is my Form" centered at the top.
- A label "First Name" followed by a text input field.
- A label "Last Name" followed by a text input field.
- A label "Marital Status" followed by a dropdown menu.

The dropdown menu is currently open, showing two options: "Single" (which is highlighted) and "Married".

- **JCheckBox**

- JCheckBox, JToggleButton and JRadioButton are state buttons on/off

- **Constructor**

- JCheckBox(String s, Icon i, Boolean state);
 - JCheckBox(String label);
 - JCheckBox(String s, Boolean state);

- **Methods**

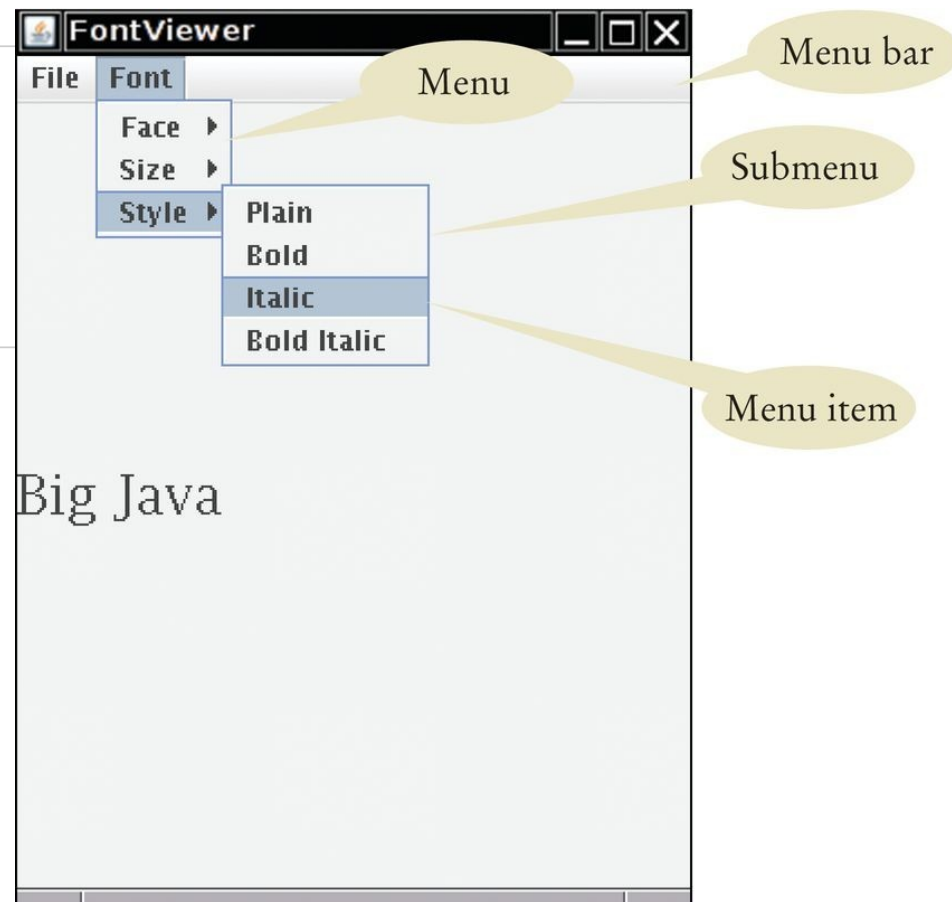
- void setSelected(boolean state);
 - boolean isSelected();
 - JRadioButton and JToggleButton are similar

- **JList**
- **Constructor**
 - `JList(String str[]);`
- **Method**
 - `int getSelectedIndex();`
 - `Object getSelectedValue();`
 - `Object []obj = getSelectedValues();`
 - `int []idx = getSelectedIndices();`
 - `setListData(ListItems);`
- It has no scrollPane, to have scrolling JList we have to add to the container JScrollPane object.
- **Syntax**
 - `container.add(new JScrollPane(obj));` //obj is JList object
 - `setVisibleRowCount(int n);`

Menus

- A frame can contain a *menu bar*. *Menu bar* contains *menus*.
- *Menu* contains *submenus* and *menu items*.
- Menu items can be added to each menu or submenu. First, add the menu bar to the frame:

```
public class MyFrame extends JFrame
{
    public MyFrame()
    {
        JMenuBar menuBar = new JMenuBar();
        setJMenuBar(menuBar);
        . . .
    }
    . . .
}
```



Menus

```
JMenu fileMenu = new JMenu("File");
```

```
JMenu fontMenu = new JMenu("Font");
```

```
menuBar.add(fileMenu);
```

```
menuBar.add(fontMenu);
```

```
JMenuItem exitItem = new JMenuItem("Exit");
```

```
fileMenu.add(exitItem);
```

```
JMenu styleMenu = new JMenu("Style");
```

```
fontMenu.add(styleMenu); // A submenu
```

```
ActionListener listener = new ExitItemListener();
```

```
exitItem.addActionListener(listener);
```

Use a separate method for each menu or set of related menus:

```
public JMenu createFaceMenu(){
```

```
    JMenu menu = new JMenu("Face");
```

```
    menu.add(createFaceItem("Serif"));
```

```
    menu.add(createFaceItem("SansSerif"));
```

```
    menu.add(createFaceItem("Monospaced"));
```

```
    return menu;
```

```
}
```

Menus

createFaceItem method needs to set the font face:

- Set the current face name.
- Make the new font from the current face, size, and style, and apply it to label.

Create a FaceItemListener class to listen for face item name

actions:

```
class FaceItemListener implements ActionListener {  
    private String name;  
    public FaceItemListener(String newName) {  
        name = newName; }  
    public void actionPerformed(ActionEvent event)  
    {  
        faceName = name; // Sets an instance variable of the frame class setLabelFont();  
    }  
}
```

Install a listener object with the appropriate name:

```
public JMenuItem createFaceItem(String name){  
    JMenuItem item = new JMenuItem(name);  
    ActionListener listener = new FaceItemListener(name);  
    item.addActionListener(listener);  
    return item;  
}
```

Menus

- Better to make a local inner class in the createFaceItem method. actionPerformed method can access the name parameter variable directly (rather than passing it).

```
public JMenuItem createFaceItem(final String name)
// Final variables can be accessed from an inner class method
{
    class FaceItemListener implements ActionListener
    // A local inner class
    {
        public void actionPerformed(ActionEvent event)
        {
            facename = name; // Accesses the local variable name setLabelFont();
        }
    }
    JMenuItem item = new JMenuItem(name);
    ActionListener listener = new FaceItemListener(name);
    item.addActionListener(listener);
    return item;
}
```

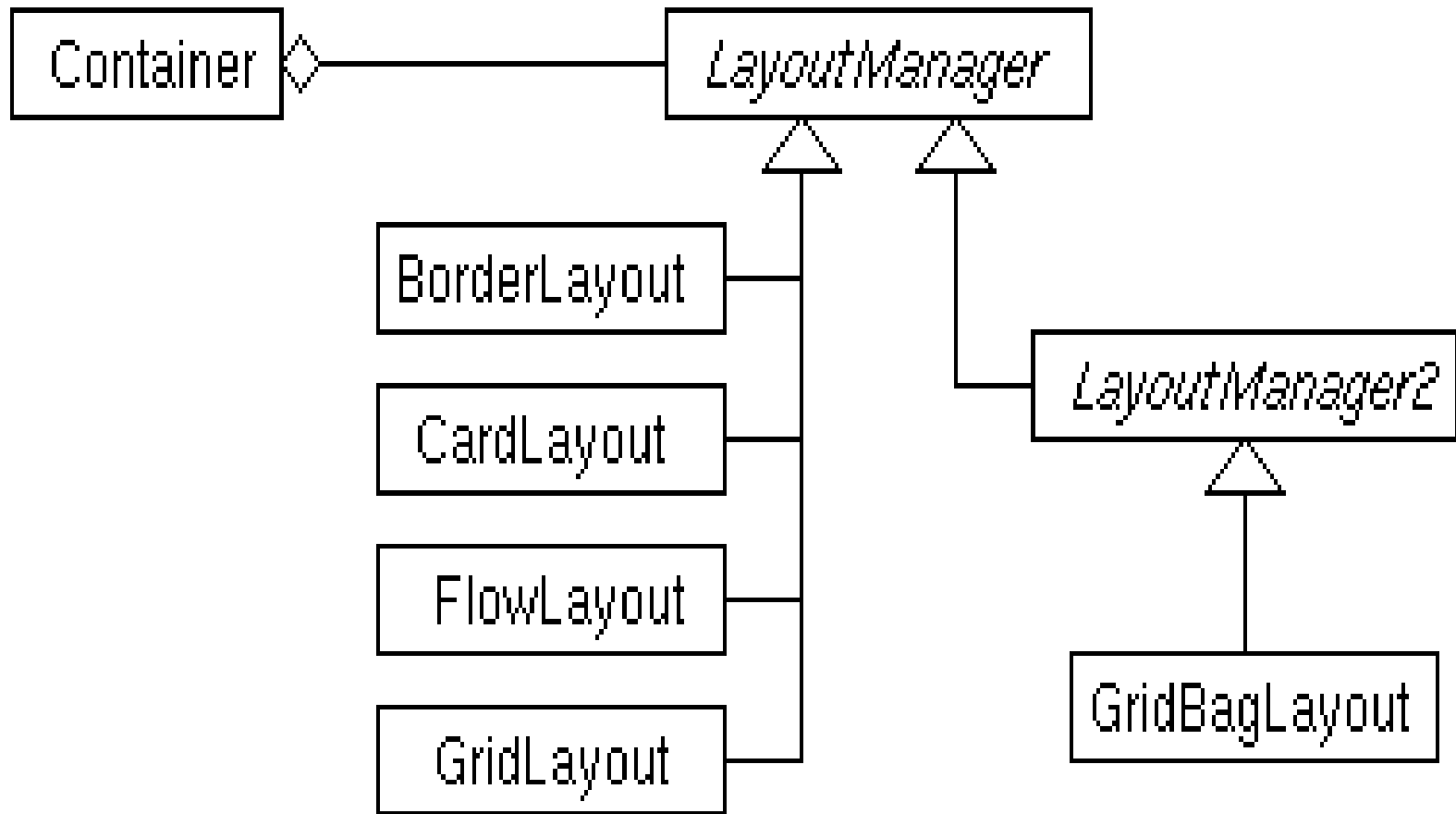
Same strategy used for the createSizeItem and createStyleItem methods.

Layout Managers

- Associated with containers
- provide a level of abstraction to automatically map your user interface on all windowing systems
- Automate the layout of elements
 - When elements are added to the container
 - When the window is resized
 - automatically adjust the positions and sizes of the elements.
- Layout managers control:
 - Where components appear
 - What sizes they are
 - How they react when they are resized

- Each container has a layout manager to arrange the UI components within the container
- Layout managers are:
 - FlowLayout
 - GridLayout
 - BorderLayout
 - CardLayout
 - GridBagLayout

Hierarchy of Layout Managers



- **Flow Layout**

- the component objects appear in order in which they are added to the container from left to right until no more component will fit on a row.
- It then moves to the next row and continue going left to right.

Constructors

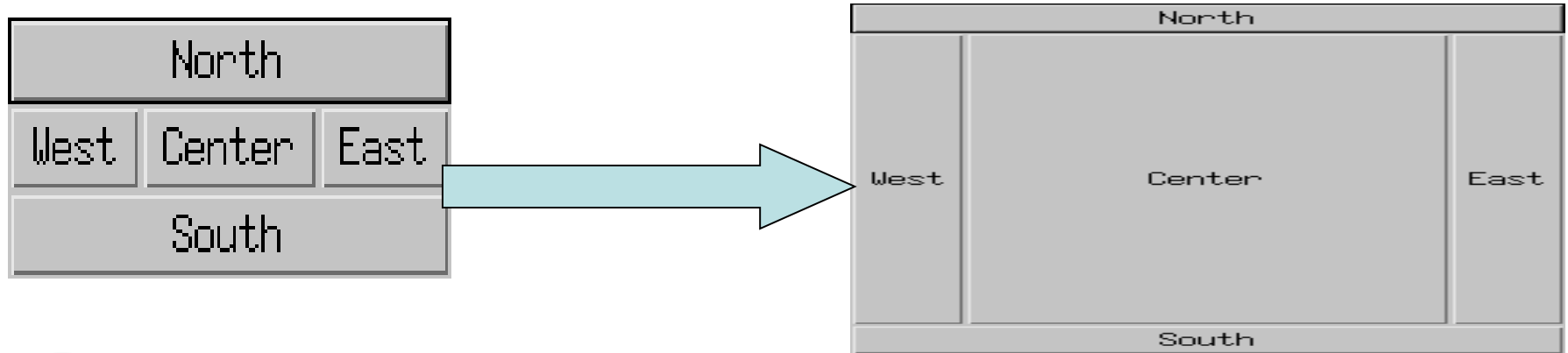
- `FlowLayout()`;
- `FlowLayout(int align)`; //align is a constant left, center or right
- `FlowLayout(int align, int hSpace, int VSpace)`;

Method:

- `setAlignment(int align);` //FlowLayout.LEFT,
 .CENTER, //RIGHT
- JFrame uses Flow layout by default.

- **BorderLayout**

- It uses five areas to hold components: **NORTH**, **SOUTH**, **EAST**, **WEST**, and **CENTER**.
- All extra space is placed in the center area.



- **Constructor:**

- `BorderLayout(int n, int m);` `//n pixel gaps`

- **Methods**

- `void add(Component cp, int region);`
- **Example:** `cont.add(btn, BorderLayout.CENTER);`

- **GridLayout**

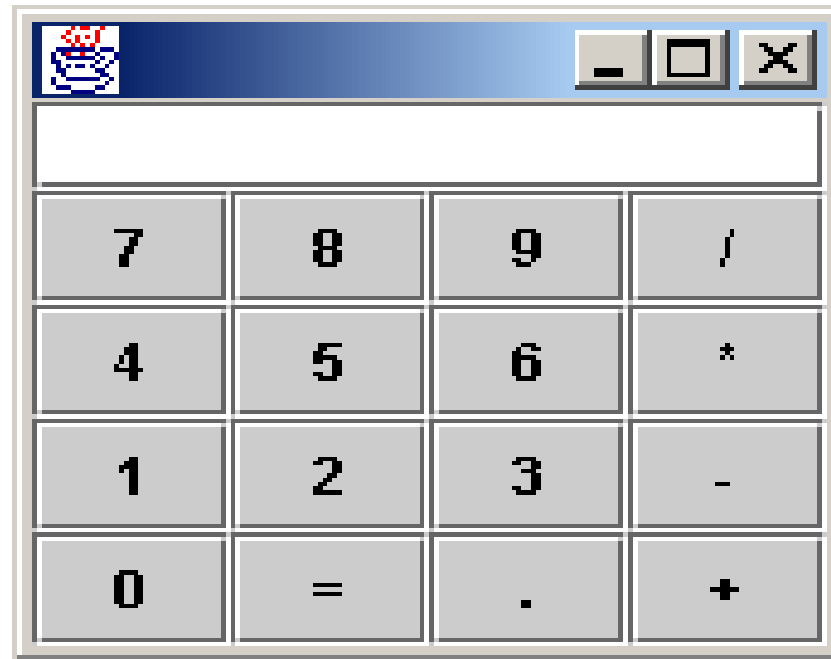
- Creates a grid structure on the container
- Every component in a GridLayout has the same width and height.
- Components are placed in boxes in a simple table arrangement. Specify the size (rows then columns) of the grid.

- **Constructor:**

- GridLayout(int row, int col);
- GridLayout(int row, int col, int hSpace, int VSpace);

- Components are added to a GridLayout starting at the top-left cell of the grid and proceeding left to right until the row is full

- **Eg.** new GridLayout(4,3);
- **Example:** Use GridLayout to write a calculator program



```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
class Calculator extends JFrame{
    JButton btn[]=new JButton[16];
    JTextField txtResult;
    Calculator(){
        String caption[]={ '7', '8', '9', '/', '4', '5', '6', '*', '1',
                            '2', '3', '-', '0', '=', '.', '+' };
        //setSize(200,200);
        for(int i=0;i<16;i++)
            btn[i]=new JButton(String.valueOf(caption[i]));
        txtResult=new JTextField();
        MyListener ml=new MyListener();
        for(int i=0;i<16;i++)
            btn[i].addActionListener(ml);
    }
}
```

```

        Container cp=getContentPane();
        cp.setLayout(new BorderLayout());
        cp.add(txtResult,BorderLayout.NORTH);
        JPanel jp=new JPanel();
        cp.add(jp,BorderLayout.CENTER);
        jp.setLayout(new GridLayout(4,4));
        for(int i=0;i<16;i++)
            jp.add(btn[i]);
        pack();
    }
    public static void main(String args[]){
        Calculator cl=new Calculator();
        cl.setVisible(true);
    }
}
class MyListener implements ActionListener{
    public void actionPerformed(ActionEvent ae){
    }
}
}

```

The Grid Bag Layout

- The grid bag layout is the mother of all layout managers.
 - the rows and columns can have variable sizes.
 - You can join adjacent cells to make room for larger components.
 - The components need not fill the entire cell area, and you can specify their alignment within cells.

- To describe the layout to the grid bag manager, use the following procedure:
 - Create an object of type GridBagLayout.
 - You don't tell it how many rows and columns the underlying grid has.
 - Instead, the layout manager will try to guess it from the information you give it later.
 - Set this GridBagLayout object to be the layout manager for the component.
 - For each component, *create an object of type GridBagConstraints*.
 - Set field values of the GridBagConstraint object to specify how the components are laid out within the grid bag.
 - Finally, add each component with its constraints by using the call
 - **add(component, constraints);**

- Example

```
GridBagLayout layout = new GridBagLayout();  
panel.setLayout(layout);  
GridBagConstraints constraints = new GridBagConstraints();  
constraints.weightx = 100;  
constraints.weighty = 100;  
constraints.gridx = 0;  
constraints.gridy = 2;  
constraints.gridwidth = 2;  
constraints.gridheight = 1;  
panel.add(component, constraints);
```

- The **gridx**, **gridy**, **gridwidth**, and **gridheight** constraints define where the component is **located in the grid**.

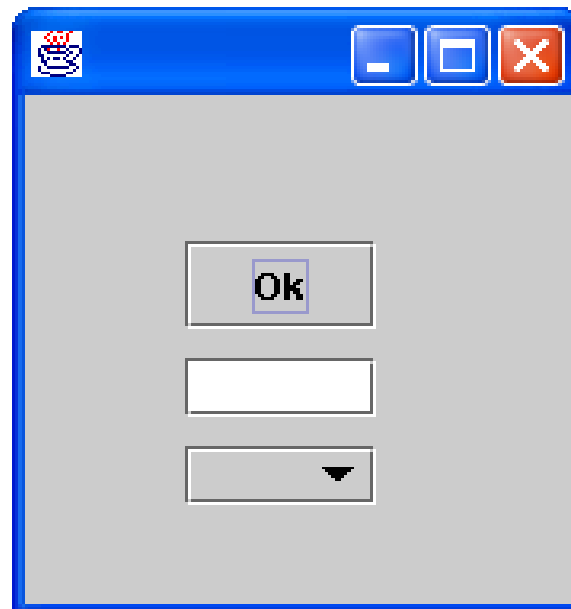
- **gridx** - The column in which the component will be placed.
- **gridy** - The row in which the component will be placed
- **gridwidth** - The number of columns the component occupies
- **gridheight** - The number of rows the component occupies
- **weightx** - The portion of extra space to allocate horizontally. The components can become wider when extra space is available
- **weighty** - The portion of extra space to allocate vertically. The components can become taller when extra space is available.
- If you set the **weight to** 0, then the area never grows or shrinks beyond its initial size in that direction.

- **The Null Layout Manager**

- If you set the layout manager in a container to **null**,
- you can explicitly set the **sizes and positions** of the components.

```
class NullLayout extends JFrame{  
    JButton btn;  
    JTextField txt;  
    JComboBox jcb;  
    NullLayout(){  
        btn = new JButton("Ok");  
        txt = new JTextField();  
        jcb = new JComboBox();  
        Container cp = getContentPane();  
        cp.add(btn);  
        cp.add(txt);  
        cp.add(jcb);  
        cp.setLayout(null);  
    }  
}
```

```
        btn.setBounds(50, 50, 60, 30);  
        txt.setBounds(50, 90, 60, 20);  
        jcb.setBounds(50, 120, 60, 20);  
    }  
    public static void main(String args[]){  
        NullLayout nl = new NullLayout();  
        nl.setVisible(true);  
    }  
}
```



The Java Event Handling

- **Event-Driven Programming**
- An **event**: a type of signal to the program that something has happened
- The event is generated by external user actions such as
 - mouse movements, mouse button clicks, and keystrokes, or by the operating system, such as a timer
- The GUI component on which an event is generated is called the **source object**
- Programmer writes code to respond to the various events that may occur.

- **The Delegation Event Model**

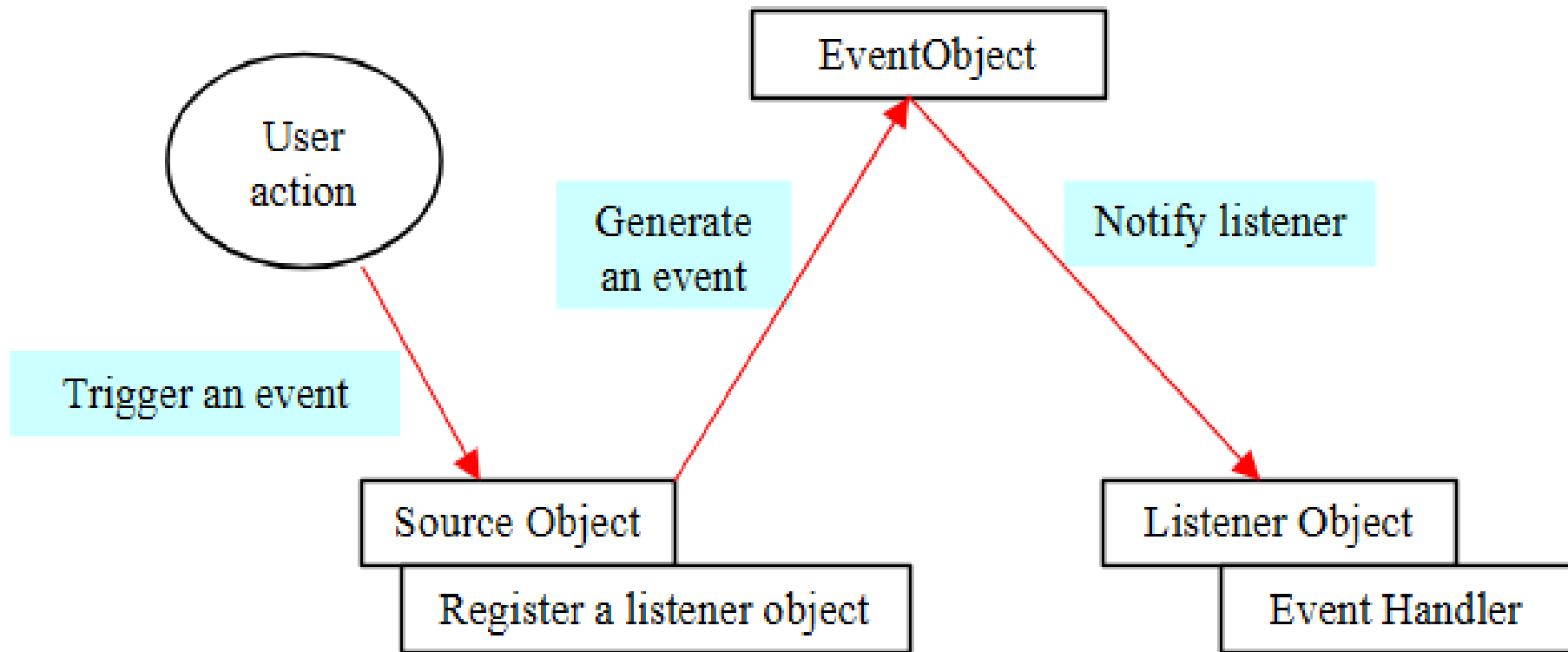
- lies at the heart of Java's event handling system
- Defines a standard and consistent mechanism to generate and process events
- The main advantage is
 - the **application logic** that process the event is cleanly separated from the user **interface logic** that generates those events.
- Events are supported by the **java.awt.event**.
Package
- There are three objects active in Delegation Event Model
 - event sources
 - event objects
 - event listeners

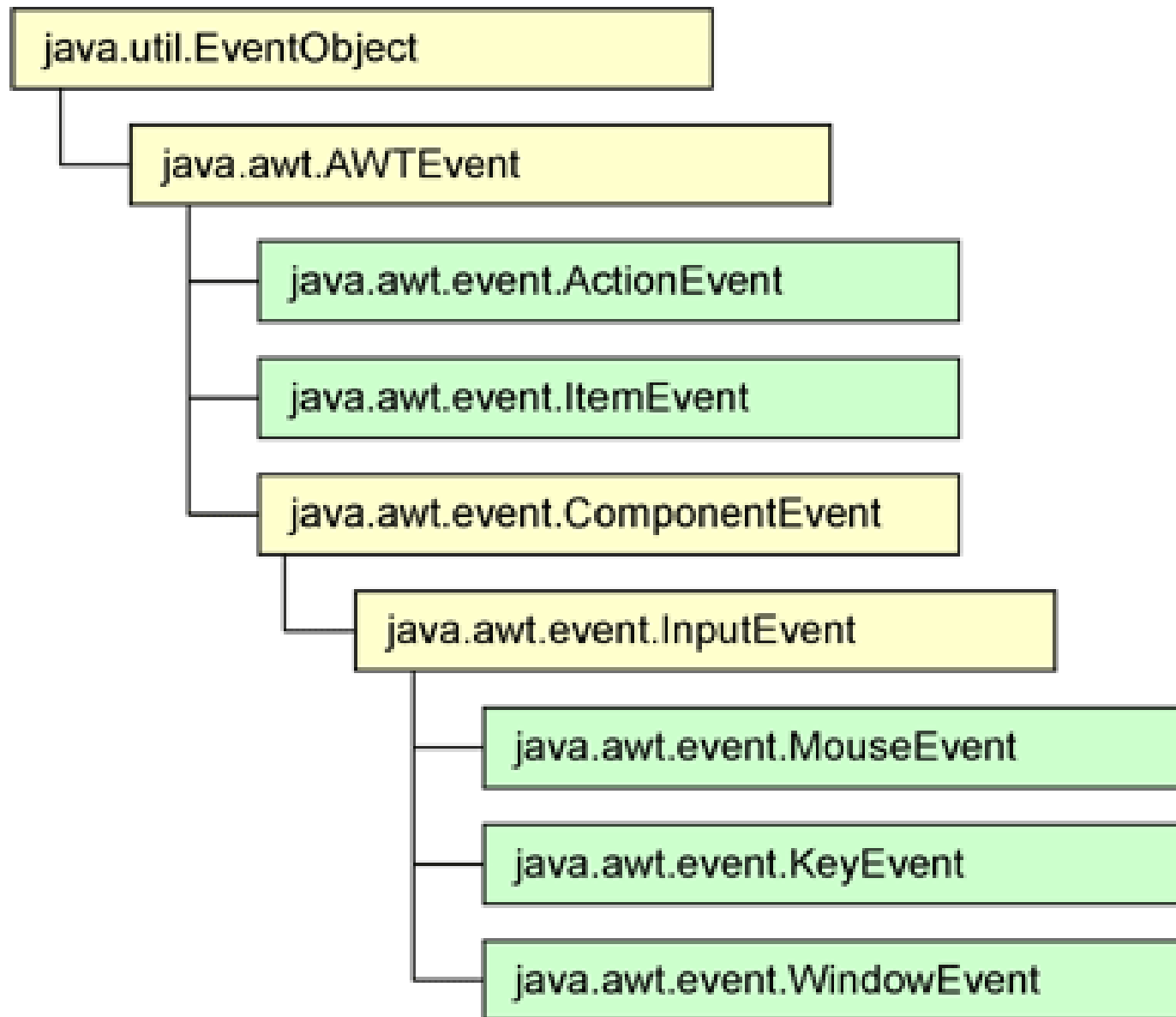
- **Event Sources**

- Events are generated when the objects internal states are changed in some way
- A source may generate more than one event.
- Event sources are objects that generates an event: menus, buttons, text fields etc.
- Event sources have methods to add event listeners to them
 - For example: **addActionListener()**
- Event source reports on events and notifies all its listeners

- **Event Objects**

- Objects that represent a user action (e.g. mouse click)
- contain reference to source that generates the event and other event information.
- When an event happens an event source sends an event object to its event listeners
- **Event can be generated as a consequence of:**
 - **Person Interaction with the elements of GUI**
 - Pressing a button, Entering a character, Selecting an item in a list, Click a mouse
 - **System generated events**
 - Timer, System Failure
- **EventObject** is the superclass
 - **ActionEvent, MouseEvent**, etc. are the subclasses that we use





- **Event Listeners**

- are objects that respond when an event occurs
- Have two major requirements.
 - It must have been registered with one or more sources to receive notifications about specific types of events.
 - It must implements methods to receive and process these notifications
- Any event listener is specific to an event source.
 - For example, you'd have one kind of event listener to respond to the click of a button on your mouse, and another to respond to the press of a key on your keyboard.
- **Listeners are interfaces, not classes**
 - Class MyButtonListener implements ActionListener
{...}

Listeners for JTextFields

- An ActionListener listens for someone hitting the Enter key
- An ActionListener requires this method:
`public void actionPerformed (ActionEvent e)`
- You can use `getText( )` to get the text
- A TextListener listens for any and all keys
- A TextListener requires this method:
`public void textValueChanged(TextEvent e)`

- User actions like clicking a button are modelled by an **ActionEvent**, and sent to an **ActionListener**.
- Window actions like clicking a close or minimize box are **WindowEvents**, sent to **WindowListeners**.
- Mouse actions like pressing the mouse is modelled in a **MouseEvent**, and is sent to a **MouseListener**.
- Mouse movement is modelled in a **MouseEvent**, and is sent to a **MouseMotionListener**.
- Key presses are modelled in a **KeyEvent**, and is sent to a **KeyListener**.

- **A source must register listeners**

- Each of the event has its own registration method:

- **Syntax:**

- <**Source** | . add<Type|Listener (<Type|Listener el);

- **Type** is the name of event

- **el** reference to the event listener

- **Example:**

- add**Key**Listener()

- addMouseListener()

- A source also provides a method to 'unregister' the listeners.

- **Syntax:**

- public void remove<type|Listener(<Type|Listener tl);

- **Example:**

- If the listener class is different class, we have to do

```
ListenerClass lc = new ListenerClass();
```

```
JButton btnTest = new JButton("Test");
```

```
btnTest.addActionListener(lc);
```

- if the Listener class is the class that contains the source object, then the reference to the Listener object will be **'this'**

```
btnTest.addActionListener(this);
```

- The Listener must implement the appropriate Listener interface and define methods to receive and process notifications

- **Example**

- to Listen and process the **ActionEvent** generated by the source button, the Listener class must implement ActionListener interface
- The method to be implemented is **actionPerformed**.

```
class ListenerClass implements ActionListener{  
    public void actionPerformed (ActionEvent ae){  
        ae.getSource();//returns source object reference  
        ae.getActionCommand(); //returns caption of  
        the  
        //source(String)  
    }  
}
```

- **ActionEvent**

- Methods

- **String getActionCommand();** - returns String(text) which is the caption on the source
 - In a Button click, TextField focus on the TextField and pressing Enter
 - **Object getSource();** - returns object reference of the source

- **Source that create ActionEvent**

- JButton, JTextField

- Listener interface that the Listener class must implement is **ActionListener**

- Method that the Listener should implement is
public void actionPerformed(ActionEvent ae);

- **ItemEvent**

- Occurred by sources **JCheckBox**, **JRadioButton**, **JComboBox**.
- **JCheckBox** and **JRadioButton** have two state **SELECTED** and **DESELECTED**

`getStateChange() == e.SELECTED`

- **Methods**

- `(event object).getSource();` //returns the source object
- `(event object).getStateChange() == ItemEvent.SELECTED` or `ItemEvent.DESELECTED`
- Listener interface that the Listener class must implement is **ItemListener**
- Method that the Listener class should implement is
`public void itemStateChanged(ItemEvent ie);`

- **MouseEvent:** There are two types of Listener interfaces
 - **MouseListener:** There are five methods to be implemented

```
public void mouseClicked(MouseEvent me);
public void mouseEntered(MouseEvent me);
public void mouseExited(MouseEvent me);
public void mouseReleased(MouseEvent me);
public void mousePressed(MouseEvent me);
```
 - **MouseMotionListener:**

```
public void mouseMoved(MouseEvent me);
public void mouseDragged(MouseEvent me);
```

- **WindowEvent**

- **WindowListener** : There are 7 methods to be implemented

- windowClosing(WindowEvent)

- windowOpened(WindowEvent)

- windowIconified(WindowEvent)

- windowDeiconified(WindowEvent)

- windowClosed(WindowEvent)

- windowActivated(WindowEvent)

- windowDeactivated(WindowEvent)

- **Ways to define a Listener class**

- By making the class that creates source components a listener class

```
class MyClass extends JFrame implements ActionListener{  
    MyClass(){  
        .  
        .  
        .  
        <source>.addActionListener(this);  
        . . .  
    }  
    public void actionPerformed(ActionEvent ae){  
    }  
}
```

– Defining a separate Listener class

```
class MyClass extends JFrame {  
    MyClass(){  
        .  
        .  
        .  
        MyHandler mh = new MyHandler();  
        <source>.addActionListener(mh);  
        . . .  
    }  
}  
  
class MyHandler implements ActionListener, ItemListener{  
    . . .  
    public void actionPerformed(ActionEvent ae){  
        . .}  
    public void itemStateChanged(ItemEvent ie){ . .  
        .}  
}
```

– Define Inner Class

```
class MyClass extends JFrame {  
    MyClass(){  
        .  
        .  
        .  
        MyHandler mh = new MyHandler();  
        <source>.addActionListener(mh);  
        . . .  
    }  
    private class MyHandler implements  
        ActionListener{  
        public void actionPerformed(ActionEvent ae)  
        { . . . }  
    }  
}
```

- An Inner class is allowed to directly access its outer class's variables and methods.

– **Define Anonymous Inner classes**

```
class MyClass extends JFrame{  
    ...  
    MyClass() {  
        ...  
        <source>.addActionListener(new  
        ActionListener(){  
            public void actionPerformed(ActionEvent ae){  
                ...  
            }  
        });  
        <source>.addWindowListener(new  
        WindowAdapter(){  
            public void  
            WindowClosing(WindowEvent ae){  
                dispose();  
                System.exit(0);  
            }  
        });  
    }  
}
```

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
class BackColor extends JFrame implements ActionListener{
    JButton btnCyan;
    JButton btnBlack;
    JButton btnCancel;
    BackColor(String str){
        btnCyan = new JButton("Cyan");
        btnBlack = new JButton("Black");
        btnCancel = new JButton("Exit");
        Container cp = getContentPane();
        setSize(200, 300);
        cp.setLayout(new FlowLayout());
        cp.add(btnCyan);
        cp.add(btnBlack);
```

```
    cp.add(btnCancel);  
    btnCancel.addActionListener(this);  
    btnBlack.addActionListener(this);  
    btnCyan.addActionListener(this);  
}  
public void actionPerformed(ActionEvent ae){  
    if(ae.getSource() == btnCyan)  
        getContentPane().setBackground(Color.cyan);  
    else if(ae.getSource() == btnBlack)  
        getContentPane().setBackground(Color.black);  
    else{  
        dispose();  
        System.exit(0);  
    }  
}  
public static void main(String args[]){  
    BackColor b = new BackColor("My Frame");  
    b.setVisible(true);  
}}
```

Java ActionListener Example: On Button click

- `import java.awt.*;`
- `import java.awt.event.*;`
- `//1st step`
- `public class ActionListenerExample implements ActionListener{`
- `public static void main(String[] args) {`
- `Frame f=new Frame("ActionListener Example");`
- `final TextField tf=new TextField();`
- `tf.setBounds(50,50, 150,20);`
- `Button b=new Button("Click Here");`
- `b.setBounds(50,100,60,30);`
- `//2nd step`
- `b.addActionListener(this);`
- `f.add(b);f.add(tf);  f.setSize(400,400);  f.setLayout(null);`
- `f.setVisible(true);  } //3rd step`
- `public void actionPerformed(ActionEvent e){`
- `tf.setText("Welcome to Javatpoint.");`
- `} }`

ActionListener using anonymous class.

- `import java.awt.*;`
- `import java.awt.event.*;`
- `public class ActionListenerExample {`
- `public static void main(String[] args) {`
- `Frame f=new Frame("ActionListener Example");`
- `final TextField tf=new TextField();`
- `tf.setBounds(50,50, 150,20);`
- `Button b=new Button("Click Here");`
- `b.setBounds(50,100,60,30);`
- `b.addActionListener(new ActionListener(){`
- `public void actionPerformed(ActionEvent e){`
- `tf.setText("Welcome to Javatpoint.");`
- `}`
- `});`
- `f.add(b);f.add(tf);`
- `f.setSize(400,400);`
- `f.setLayout(null);`
- `f.setVisible(true);`
- `} }`

- <https://stackhowto.com/actionlistener-java-swing-example/>
- <https://programming.vip/docs/java-swing-three-ways-to-implement-actionlistener-listener.html>
- <https://docs.oracle.com/javase/tutorial/displayCode.html?code=https://docs.oracle.com/javase/tutorial/uiswing/examples/layout/GridBagLayoutDemoProject/src/layout/GridBagLayoutDemo.java>
- https://www3.ntu.edu.sg/home/ehchua/programming/java/j4a_gui.html
- [https://www.mindluster.com/certificate/3099?
gclid=CjwKCAjw9e6SBhB2EiwA5myr9iPgLTyW4KTh-3yIvurwbJgmUcJJzY26S1LLfmwpYF-6eZD2XvNyxho
CgGAQAvD_BwE](https://www.mindluster.com/certificate/3099?gclid=CjwKCAjw9e6SBhB2EiwA5myr9iPgLTyW4KTh-3yIvurwbJgmUcJJzY26S1LLfmwpYF-6eZD2XvNyxhoCgGAQAvD_BwE)
- [https://study.com/academy/lesson/event-driven-programming-in-java-definition-
components.html#:~:text=In%20Java%20GUI%20applications%20using,to%20as%20being%20event%2Ddriven.](https://study.com/academy/lesson/event-driven-programming-in-java-definition-components.html#:~:text=In%20Java%20GUI%20applications%20using,to%20as%20being%20event%2Ddriven.)